IEEE 830 · SOFTWARE ENGINEERING DOCUMENT

Software Requirements Specification

A formal requirements document prepared for academic evaluation, development planning, and quality assurance

PROJECT — Student Management System

Document Title	Software Requirements Specification (SRS) — Student Management System
Document Version	1.0 (Baseline)
Document Status	Final Draft — For Faculty Review & Project Development
Prepared For	College / University Software Engineering Course Project
Applicable Standard	IEEE Std 830-1998 (Recommended Practice for SRS)
Date of Issue	14 August 2026

3	6	8	39
USER ROLES	CORE MODULES	DATA ENTITIES	FUNCTIONAL REQUIREMENTS

SUBMITTED BY

Student Name(s) _____ **Roll / Enrollment No.** _____

Class / Semester _____ **Department** _____

Guided By _____ **Academic Year** _____

Institution _____

Document Control

Revision History

Version	Date	Description	Author(s)
0.1	Week 1	Initial draft — scope, stakeholders, and preliminary feature list	Project Team
0.5	Week 2	Added functional requirements, use cases, and data model	Project Team
1.0	14 Aug 2026	Reviewed and baselined for development and faculty submission	Project Team / Guide

Distribution List

Name / Role	Purpose
Project Guide / Faculty Mentor	Review, evaluation, and sign-off
Development Team (Students)	Implementation reference
Department Evaluation Committee	Academic assessment

Document Purpose Note

How to use this document

This SRS defines **what** the Student Management System must do, not **how** it is coded internally. It is written to be directly usable for UI/UX wireframing, database/API design, test-case creation, and final project report/viva preparation. Design and implementation details (technology stack, class diagrams, schema DDL) are intentionally kept at a planning level so the development team retains flexibility during Phase 2 (design) and Phase 3 (implementation).

Project Team & Roles

Role	Name / Roll No.	Primary Responsibility
Team Lead	_____	Coordination, requirement sign-off, and final integration
Backend Developer	_____	Application/business logic layer, database, and API design
Frontend / UI Developer	_____	Screens, forms, and role-based dashboards
QA / Tester	_____	Test-case design and execution against Sections 5 and 8

Related / Reference Documents

Document	Status
Project Synopsis / Proposal	Submitted prior to this SRS
High-Level / Low-Level Design Document	To be prepared in Phase 2, based on Sections 3 and 6 of this SRS
Test Plan & Test Cases	To be prepared in Phase 3, based on Sections 5 and 8 of this SRS

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Document Conventions 4

1.3 Intended Audience and Reading Suggestions 4

1.4 Project Scope 5

1.5 Definitions, Acronyms and Abbreviations 5

1.6 References 6

2. Overall Description 6

2.1 Product Perspective 6

2.2 Product Features (Summary) 7

2.3 User Classes and Characteristics 7

2.4 Operating Environment 8

2.5 Design and Implementation Constraints 8

2.6 Assumptions and Dependencies 9

3. System Architecture Overview 9

4. Use Case Model 11

4.1 Actors 11

4.2 Use Case Summary Table 11

4.3 Detailed Use Case Specifications 12

5. Functional Requirements 14

5.1 Module: User Authentication & Access Control 14

5.2 Module: Student Management 15

5.3 Module: Course Management 16

5.4 Module: Attendance Management 17

5.5 Module: Grade / Result Management 18

5.6 Module: Dashboard & Reports 19

6. Data Model (Logical) 20

6.1 Entity-Relationship Diagram 20

6.2 Entity Descriptions 21

6.3 Relationship Summary 22

7. External Interface Requirements 23

7.1 User Interfaces 23

7.2 Hardware Interfaces 24

7.3 Software Interfaces 24

7.4 Communication Interfaces 24

8. Non-Functional Requirements 25

9. Other Requirements 27

10. Appendix A — Future Scope 27

11. Appendix B — Approval 28

SECTION 1

Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the **Student Management System (SMS)**, a web-based academic administration application intended to centralize and simplify the management of student records, course information, attendance, and academic grades. The document is written to serve as the single point of reference between the project stakeholders (faculty guide, evaluators) and the student development team throughout requirement analysis, design, implementation, testing, and final evaluation.

1.2 Document Conventions

Throughout this document the following conventions are used:

- Functional requirements are labelled using the format **FR-<Module Code>-<Number>** (e.g., FR-STU-01).
- Priority levels follow the **MoSCoW**-style scale shown as badges: HIGH must be implemented for a working system, MEDIUM should be implemented if time permits, LOW is a stretch/optional enhancement.
- The keyword **shall** indicates a mandatory requirement; **should** indicates a recommended but non-mandatory requirement.
- Entity names in the data model are written in Title Case; attribute names use CamelCase.

1.3 Intended Audience and Reading Suggestions

Audience	Recommended Sections
Faculty Guide / Evaluator	Sections 1, 2, 4, 8, 9 — for scope, correctness, and academic rigor assessment
Student Development Team (Coders)	Sections 3, 5, 6, 7 — for architecture, module logic, database, and interface design
UI/UX Designer (team member)	Sections 2.3, 4, 7.1 — for user classes, workflows, and screen requirements
Tester / QA (team member)	Sections 5, 8 — for writing test cases against functional and non-functional requirements

1.4 Project Scope

The Student Management System will digitize the core academic record-keeping activities of a single college department or a small institution. It replaces manual registers and scattered spreadsheets with one authenticated, role-based web application. The system is scoped to be realistic for a semester-long student project team of 2–5 members, while still demonstrating professional software engineering practice: layered architecture, relational data modelling, input validation, role-based access control, and report generation.

In scope for the initial release (Version 1.0):

- Secure login and role-based access for Admin, Faculty, and Student users.
- Creating, updating, viewing, and deactivating student profiles.
- Managing courses/subjects and assigning them to faculty and departments.
- Recording and viewing daily/session-wise attendance per course.
- Entering and viewing marks/grades per course and generating a grade report (report card).
- Dashboards and printable/exportable summary reports for each role.

Out of scope for Version 1.0 (see Section 10 — Future Scope):

- Online fee payment and finance/accounting modules.
- Hostel, library, and transport management.
- Automated timetable generation and mobile native applications.
- Biometric/RFID-based automatic attendance capture (manual/marking-based entry only in V1.0).

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SMS	Student Management System — the application described in this document
SRS	Software Requirements Specification
Admin	Administrator role with full system privileges (also referred to as Registrar/Office Staff)
Faculty	Teaching staff member responsible for one or more courses
FR	Functional Requirement

NFR	Non-Functional Requirement
CRUD	Create, Read, Update, Delete — basic data operations
RBAC	Role-Based Access Control
ER Diagram	Entity-Relationship Diagram — a visual model of the database structure
API	Application Programming Interface
UI/UX	User Interface / User Experience

1.6 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Institution's academic curriculum structure and semester/grading policy (as provided by the college).
- Standard relational database design principles (Third Normal Form / 3NF) used for Section 6.

SECTION 2

Overall Description

2.1 Product Perspective

The Student Management System is a **new, self-contained web application**; it does not replace or interface with any existing legacy institutional software. It is designed as a standard 3-tier web system consisting of a browser-based front end, a server-side application layer that enforces business rules, and a relational database that persists all academic records. A conceptual view of this architecture is provided in Section 3.

2.2 Product Features (Summary)

At a high level, the system provides six cohesive feature groups, each expanded into detailed functional requirements in Section 5:

Feature Group	Summary
Authentication & Access Control	Secure login, password management, and role-based screen/action visibility for Admin, Faculty, and Student.
Student Management	Register, edit, search, view, and archive student profiles and enrollment details.
Course Management	Define courses/subjects, assign credits, and allocate them to departments and faculty.
Attendance Management	Mark and review per-session attendance; calculate attendance percentage per student per course.
Grade Management	Record marks per exam type, auto-calculate letter grades, and generate report cards.
Dashboard & Reports	Role-specific summary dashboards and exportable (PDF/CSV) academic reports.

2.3 User Classes and Characteristics

User Class	Technical Expertise	Characteristics & System Usage
Administrator	Moderate	Office/academic staff who own the system. Creates and manages student, faculty, and course records; configures departments and academic sessions; has full read/write access; monitors overall reports. Expected to use the system daily.
Faculty	Basic–Moderate	Teaching staff assigned to specific courses. Marks attendance for their sessions and enters/updates marks for their courses only. Cannot edit student personal data or courses they are not assigned to.
Student	Basic	End-user consumer of academic information. Views personal profile, enrolled courses, attendance percentage, and grade/report card in read-only mode. Cannot modify any academic record.

2.4 Operating Environment

- **Client side:** Any modern desktop or mobile web browser (Chrome, Firefox, Edge, Safari) with a minimum screen resolution of 1024×768; the interface is responsive for tablet-size screens.
- **Server side:** A web/application server capable of hosting the chosen backend framework (e.g., Node.js/Express, Django, or Spring Boot — final selection at design stage) on Windows/Linux.
- **Database:** A relational database management system such as MySQL, PostgreSQL, or SQL Server.
- **Network:** Standard HTTPS over TCP/IP; deployable on a college intranet or a public cloud/student hosting platform.

2.5 Design and Implementation Constraints

- The system must be deliverable within a single academic semester by a small student team; therefore the feature set in Section 1.4 is intentionally focused rather than exhaustive.
- The database must be a relational (SQL) system and normalized to at least Third Normal Form (3NF) to avoid data redundancy.
- All passwords must be stored using a one-way cryptographic hash (e.g., bcrypt) — never in plain text.
- The application must be built using technologies available under free/open-source licences suitable for academic use.
- The system is designed for a single institution/department instance (not multi-tenant) in Version 1.0.

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms in a browser; no specialized training is assumed.
- Academic structure (departments, semesters, grading scale) is configured by the Administrator before day-to-day use begins.
- Internet/intranet connectivity is available wherever the system is accessed; the system is not designed for offline use in V1.0.
- A single Administrator account is provisioned during initial setup ("seed" admin) to bootstrap the system.

SECTION 3

System Architecture Overview

The Student Management System follows a standard **layered (3-tier) architecture**, which keeps the interface, business rules, and data storage independent of one another. This separation makes the system easier to build in parallel as a team, easier to test, and easier to extend in future phases (see Section 10).

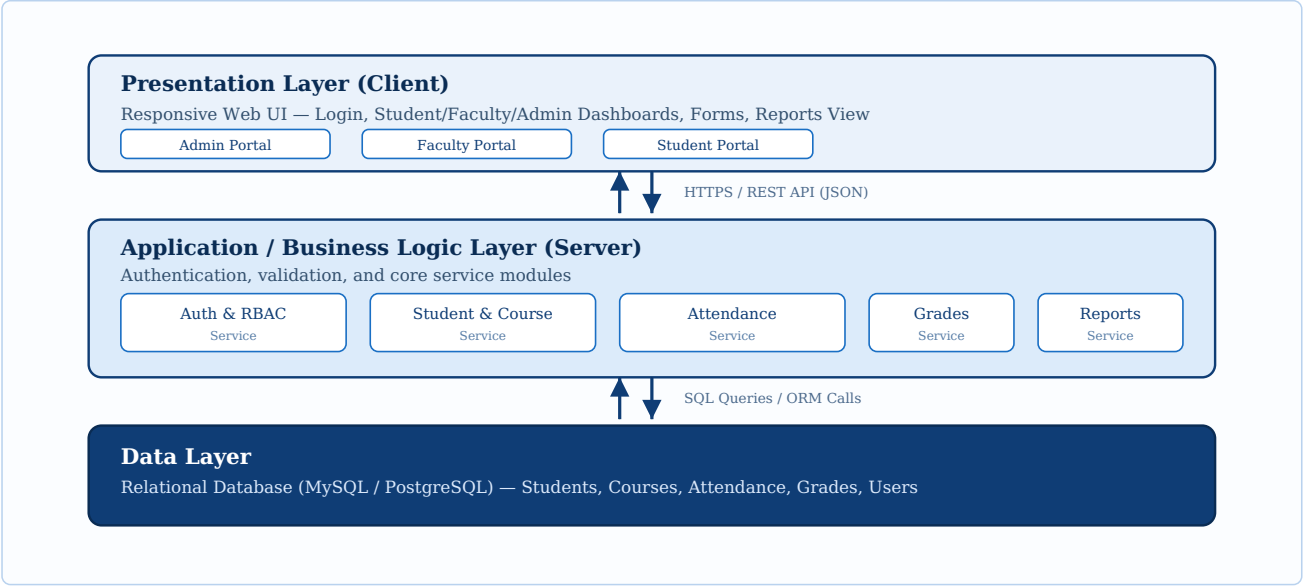


Figure 3.1 — Three-Tier System Architecture of the Student Management System

Communication between the presentation layer and the application layer occurs through a set of well-defined REST-style API endpoints exchanging JSON. Each service module in the application layer is responsible for one functional area and performs input validation and business-rule enforcement before reading from or writing to the database. This modular boundary also maps directly onto the functional requirement modules defined in Section 5, making the architecture easy to divide among team members (e.g., one member owns the Attendance service and its UI, another owns Grades, etc.).

SECTION 4

Use Case Model

4.1 Actors

Actor	Description
Administrator	Primary actor with full system access; manages master data (students, faculty, courses, departments) and oversees reports.
Faculty	Secondary actor limited to the courses assigned to them; marks attendance and enters grades.
Student	Secondary actor with read-only access to their own academic data.
System	Supporting actor that performs automated actions such as grade calculation and attendance percentage computation.

4.2 Use Case Summary Table

ID	Use Case	Primary Actor	Goal
UC-01	Login / Logout	All Users	Authenticate to access role-appropriate functionality
UC-02	Manage Student Record	Administrator	Add, edit, view, or deactivate a student profile
UC-03	Manage Course	Administrator	Create/edit a course and assign it to a faculty and department
UC-04	Enroll Student in Course	Administrator	Link a student to one or more courses for a semester
UC-05	Mark Attendance	Faculty	Record attendance for a course session
UC-06	View Attendance Report	Faculty / Student / Admin	Review attendance percentage by student or by course
UC-07	Enter / Update Grades	Faculty	Record marks for an assessment and let the system compute a letter grade
UC-08	View Report Card	Student / Admin	View a consolidated grade report for a student
UC-09	View Dashboard	All Users	See a role-specific summary on login (counts, alerts, quick links)
UC-10	Manage User Accounts	Administrator	Create accounts, assign roles, reset passwords

4.3 Detailed Use Case Specifications

UC-02 — Manage Student Record

Actor	Administrator
Precondition	Administrator is logged in and authorized.
Main Flow	1. Administrator navigates to the Student Management screen. 2. Administrator selects "Add New Student" and fills in personal, contact, and department details. 3. System validates all mandatory fields and checks for duplicate enrollment/roll numbers. 4. System saves the record and displays a confirmation message. 5. Administrator may search, edit, or deactivate an existing student from the same screen.
Alternate Flow	If validation fails (missing field, duplicate ID, invalid email/phone format), the system highlights the offending field(s) and the record is not saved.
Postcondition	Student record is created/updated in the database and is immediately searchable.

UC-05 — Mark Attendance

Actor	Faculty
Precondition	Faculty is logged in and assigned to at least one course with enrolled students.
Main Flow	1. Faculty selects a course and a session date. 2. System displays the enrolled-student roster for that course, defaulting each student to "Present". 3. Faculty toggles status (Present / Absent / Late) for individual students as needed and submits. 4. System stores one attendance record per student per session and recalculates the running attendance percentage.
Alternate Flow	If attendance for that course and date already exists, the system loads it in edit mode instead of creating a duplicate.
Postcondition	Attendance is persisted and immediately visible to the Student and the Administrator.

UC-07 — Enter / Update Grades

Actor	Faculty
Precondition	Faculty is assigned to the course and an assessment type (e.g., Mid-Term, Final, Assignment) is selected.
Main Flow	1. Faculty selects course and assessment type, then enters marks obtained for each student against the maximum marks. 2. System validates that marks obtained does not exceed maximum marks. 3. On save, the system computes a letter grade using the configured grading scale and stores the result. 4. The student's overall report card is automatically refreshed to include the new marks.
Postcondition	Grade record saved; visible to the Student (read-only) and Administrator.

SECTION 5

Functional Requirements

Each requirement below is uniquely identified, prioritized, and phrased using "shall" to indicate a mandatory, testable system behaviour.

5.1 Module: User Authentication & Access Control

FR-AUTH-01 **User Login**

HIGH Actors: All Users

The system shall allow a registered user to log in using a username/email and password, and shall redirect the user to a role-specific dashboard upon successful authentication.

FR-AUTH-02 **Role-Based Access Control**

HIGH Actors: System

The system shall restrict every screen and action to the roles authorized for it (Admin, Faculty, Student) and shall reject unauthorized requests with an appropriate error.

FR-AUTH-03 **Password Security**

HIGH Actors: System

The system shall store all passwords using a salted cryptographic hash and shall never display or transmit passwords in plain text.

FR-AUTH-04 **Change / Reset Password**

MEDIUM Actors: All Users, Administrator

The system shall allow a logged-in user to change their own password, and shall allow the Administrator to reset the password of any user account.

FR-AUTH-05 **Session Timeout**

LOW Actors: System

The system should automatically log out a user session after a configurable period of inactivity.

5.2 Module: Student Management

FR-STU-01 **Add Student Record**

HIGH Actors: Administrator

The system shall allow the Administrator to create a new student profile capturing name, date of birth, gender, contact number, email, address, department, and enrollment year.

FR-STU-02 **Edit / View Student Record**

HIGH Actors: Administrator

The system shall allow the Administrator to view and edit any field of an existing student profile, and shall allow Students to view (read-only) their own profile.

FR-STU-03 **Search / Filter Students**

HIGH Actors: Administrator, Faculty

The system shall allow searching for students by name, enrollment number, or department, and filtering by department, semester, or status (Active/Inactive).

FR-STU-04 **Deactivate Student**

MEDIUM Actors: Administrator

The system shall allow the Administrator to mark a student as Inactive (e.g., on graduation or withdrawal) instead of permanently deleting historical academic records.

FR-STU-05 **Data Validation**

HIGH Actors: System

The system shall validate that email addresses and phone numbers follow a correct format, and shall prevent duplicate enrollment numbers before saving a student record.

5.3 Module: Course Management

FR-CRS-01 **Create / Edit Course**

HIGH Actors: Administrator

The system shall allow the Administrator to define a course with a unique course code, name, credit value, department, and semester.

FR-CRS-02 **Assign Faculty to Course**

HIGH Actors: Administrator

The system shall allow the Administrator to assign one faculty member as the instructor of record for a course.

FR-CRS-03 **Enroll Students in Course**

HIGH Actors: Administrator

The system shall allow the Administrator to enroll one or more students into a course for a given academic term, forming the class roster used by Attendance and Grades.

FR-CRS-04 **View Course List**

MEDIUM Actors: All Users

The system shall display a list of courses relevant to the logged-in user's role (all courses for Admin, assigned courses for Faculty, enrolled courses for Student).

5.4 Module: Attendance Management

FR-ATT-01 **Mark Session Attendance**

HIGH Actors: Faculty

The system shall allow Faculty to mark each enrolled student's attendance status (Present, Absent, or Late) for a specific course session and date.

FR-ATT-02 **Edit Attendance**

MEDIUM Actors: Faculty

The system shall allow Faculty to correct previously submitted attendance for a session within a configurable edit window (e.g., 7 days).

FR-ATT-03 **Calculate Attendance Percentage**

HIGH Actors: System

The system shall automatically compute each student's attendance percentage per course as (sessions attended ÷ total sessions held) × 100.

FR-ATT-04 **View Attendance**

HIGH Actors: Student, Faculty, Administrator

The system shall allow Students to view their own attendance percentage per course, and Faculty/Admin to view attendance for any course or student they are authorized to see.

FR-ATT-05 **Low Attendance Flag**

LOW Actors: System

The system should visually flag students whose attendance percentage falls below a configurable threshold (e.g., 75%).

5.5 Module: Grade / Result Management

FR-GRD-01 **Enter Marks**

HIGH Actors: Faculty

The system shall allow Faculty to enter marks obtained by each enrolled student for a defined assessment type (e.g., Assignment, Mid-Term, Final) against a maximum mark value.

FR-GRD-02 **Auto-Calculate Letter Grade**

HIGH Actors: System

The system shall automatically compute a letter grade (e.g., A, B, C, D, F) from the percentage of marks obtained using a configurable grading scale.

FR-GRD-03 **Generate Report Card**

HIGH Actors: Student, Administrator

The system shall generate a consolidated, printable report card per student per semester, listing all enrolled courses, marks, and grades, plus an overall grade point average (GPA).

FR-GRD-04 **Edit Marks**

MEDIUM Actors: Faculty

The system shall allow Faculty to correct previously entered marks, and shall log the timestamp of the most recent update for audit purposes.

5.6 Module: Dashboard & Reports

FR-RPT-01 **Role-Specific Dashboard**

HIGH Actors: All Users

The system shall present a summary dashboard immediately after login: total students/courses and quick links for Admin; assigned courses and pending attendance for Faculty; attendance percentage and latest grades for Student.

FR-RPT-02 **Export Reports**

MEDIUM Actors: Administrator, Faculty

The system shall allow attendance summaries and grade/report-card data to be exported as PDF and/or CSV files.

FR-RPT-03 **Departmental Summary Report**

LOW Actors: Administrator

The system should provide an aggregate report of average attendance and average grade per course/department for administrative review.

SECTION 6

Data Model (Logical)

The logical data model below identifies the core entities required to support the functional requirements in Section 5. It is presented at a conceptual level (entities, key attributes, and relationships) so the development team can translate it into physical database tables during the design phase without the SRS constraining specific column types.

6.1 Entity-Relationship Diagram

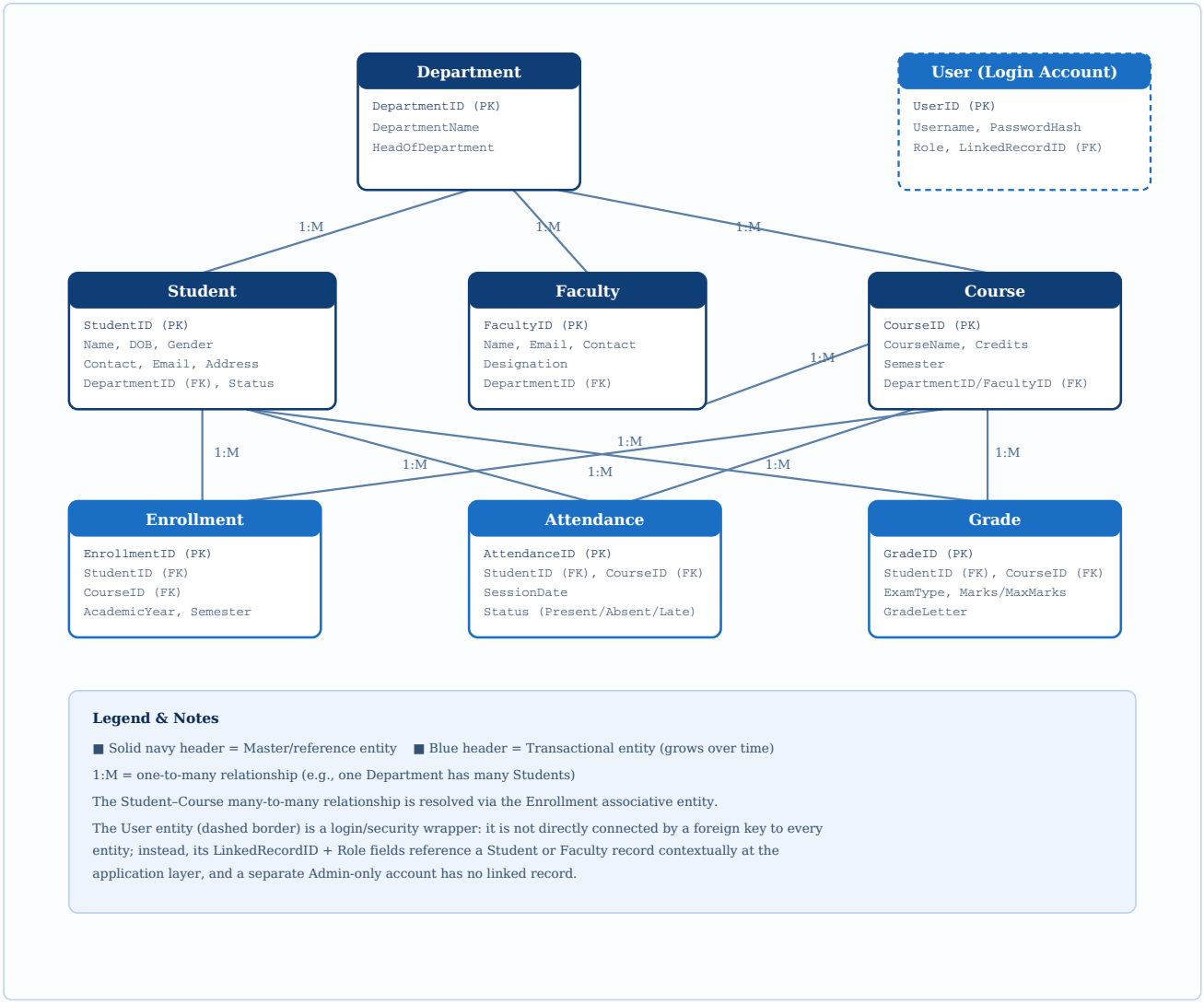


Figure 6.1 — Conceptual Entity-Relationship Diagram

6.2 Entity Descriptions

Entity	Purpose	Key Attributes
Department	Master reference data for academic departments.	`DepartmentID`, `DepartmentName`
User	Login credentials and role, decoupled from personal data.	`UserID`, `Username`, `Role`
Student	Core student profile and enrollment metadata.	`StudentID`, `Name`, `DepartmentID`
Faculty	Teaching staff profile.	`FacultyID`, `Name`, `DepartmentID`
Course	Subject/course offered by a department and taught by a faculty member.	`CourseID`, `CourseName`, `FacultyID`
Enrollment	Associative entity linking a Student to a Course for a given term (resolves the M:N relationship).	`EnrollmentID`, `StudentID`, `CourseID`
Attendance	One record per student, per course, per session date.	`AttendanceID`, `StudentID`, `CourseID`, `SessionDate`
Grade	One record per student, per course, per assessment type.	`GradeID`, `StudentID`, `CourseID`, `ExamType`

6.3 Relationship Summary

Relationship	Cardinality	Description
Department → Student	1 : M	A department has many students; a student belongs to exactly one department.
Department → Faculty	1 : M	A department has many faculty members.
Department → Course	1 : M	A department offers many courses.

Faculty → Course	1 : M	A faculty member may be assigned to teach multiple courses; each course has one primary instructor.
Student ↔ Course (via Enrollment)	M : N	A student enrolls in many courses; a course has many enrolled students.
Student → Attendance	1 : M	A student accumulates many attendance records over time.
Course → Attendance	1 : M	A course has many attendance session records.
Student → Grade	1 : M	A student accumulates many grade records across assessments.
Course → Grade	1 : M	A course has many grade records (one per student per assessment).

SECTION 7

External Interface Requirements

7.1 User Interfaces

The system provides distinct, role-appropriate screens. The following screen inventory is intended to directly seed the UI/UX wireframing task:

Screen	Primary Role(s)	Key Elements
Login Page	All	Username/email field, password field, "forgot password" link, error messaging
Dashboard (Admin)	Admin	Counts of students/faculty/courses, quick-add shortcuts, recent activity
Dashboard (Faculty)	Faculty	Assigned course list, pending-attendance alerts, quick links to mark attendance/grades
Dashboard (Student)	Student	Attendance percentage summary, latest grades, enrolled course list
Student List / Profile	Admin, Faculty (view), Student (own)	Searchable/filterable table; profile form with validation messages
Course Management	Admin	Course form, faculty assignment dropdown, enrolled-student roster
Attendance Marking	Faculty	Date picker, course selector, roster with Present/Absent/Late toggle
Grade Entry	Faculty	Course + assessment-type selector, roster with marks input and live max-marks validation
Report Card	Student (own), Admin	Printable summary table of courses, marks, grades, and GPA
Reports / Export	Admin, Faculty	Filter controls, export-to-PDF/CSV buttons

General UI guidelines: consistent blue/white institutional colour scheme, minimum 3:1 contrast for text, responsive layout down to tablet width, and clear inline validation messages next to each form field.

7.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard desktop/laptop computers or tablets with a keyboard, mouse/touch input, and a display. No direct hardware integration (e.g., biometric devices, printers) is required for Version 1.0; report/report-card printing uses the browser's standard print/PDF export capability.

7.3 Software Interfaces

Interface	Description
Web Browser	Renders the client application (HTML/CSS/JavaScript); communicates with the backend over HTTPS.
Application Server	Hosts the backend service layer and exposes REST API endpoints consumed by the client.
Database Server (RDBMS)	Persists all application data; accessed by the backend through SQL/ORM calls.
PDF/CSV Export Library	Third-party library used server-side or client-side to generate downloadable report files.

7.4 Communication Interfaces

All communication between the client and server shall occur over HTTPS. Data exchanged between the client and the REST API shall be formatted as JSON. No email/SMS gateway integration is required for Version 1.0 (optional notification features are listed under Future Scope).

SECTION 8

Non-Functional Requirements

8.1 Performance

ID	Requirement
NFR-01	The system shall load any dashboard or list screen (up to 500 records) within 3 seconds under normal load on a standard broadband/campus network connection.
NFR-02	The system shall support at least 50 concurrent users without noticeable degradation, sufficient for a single department's staff and student body.

8.2 Security

ID	Requirement
NFR-03	All data in transit shall be encrypted using HTTPS/TLS.
NFR-04	Passwords shall be stored as salted hashes; the system shall never log or display plain-text passwords.
NFR-05	The system shall enforce role-based access control at the API/service layer (not only hidden in the UI) so that a user cannot access another role's data by directly calling an endpoint.
NFR-06	The system shall validate and sanitize all user inputs on the server side to prevent SQL injection and cross-site scripting (XSS).

8.3 Reliability & Availability

ID	Requirement
NFR-07	The system shall target 99% uptime during the academic term for the deployed environment.
NFR-08	The system shall provide clear, non-technical error messages to users when an operation fails, while logging technical details for developers.
NFR-09	Critical data (student, attendance, grade records) shall be included in a regular backup routine (e.g., daily database backup).

8.4 Usability

ID	Requirement
NFR-10	A first-time user shall be able to complete a core task (e.g., marking attendance) within 3 clicks/steps from the dashboard after a brief walkthrough.
NFR-11	The UI shall be responsive and usable on screen widths from 768px (tablet) upward.
NFR-12	The system shall provide consistent navigation (persistent menu/header) and clear labelling across all screens.

8.5 Maintainability & Scalability

ID	Requirement
NFR-13	The codebase shall follow the layered architecture in Section 3, keeping UI, business logic, and data access separated to simplify future maintenance by a student team.
NFR-14	The database schema shall be normalized (3NF) to minimize redundancy and ease future feature additions.
NFR-15	The system shall be designed so that additional departments, courses, and users can be added through configuration/data entry, without code changes.

SECTION 9

Other Requirements

9.1 Legal, Compliance & Data Retention

- Student personal data shall be accessible only to authorized roles, consistent with the institution's data privacy policy.
- Academic records (attendance and grades) shall be retained for at least the duration of the student's enrollment plus any period required by

institutional record-keeping policy.

9.2 Database Requirements Summary

For quick reference during database/API planning, the physical database should implement: normalized tables mirroring the entities in Section 6.2, foreign-key constraints enforcing the relationships in Section 6.3, unique constraints on natural keys (e.g., student enrollment number, course code, username), and indexes on frequently filtered columns (StudentID, CourseID, SessionDate) to keep attendance and grade queries fast as data grows.

SECTION 10

Appendix A — Future Scope

The following enhancements are explicitly out of scope for Version 1.0 (Section 1.4) but are documented here to guide future project phases or a follow-on capstone project:

Enhancement	Description
Email/SMS Notifications	Automated alerts for low attendance or new grade postings.
Timetable Module	Class scheduling and clash detection.
Fee Management	Fee structure, payment tracking, and receipts.
Biometric/RFID Attendance	Automated attendance capture integrated with hardware devices.
Mobile Application	Native Android/iOS companion app consuming the same REST API.
Analytics Dashboard	Trend charts for attendance and performance across semesters.
Multi-Institution Support	Multi-tenant architecture to support more than one institution per deployment.

SECTION 11

Appendix B — Approval

This Software Requirements Specification has been reviewed and is approved as the baseline (Version 1.0) for the design and development of the Student Management System project.

Role	Name	Signature	Date
Project Guide / Faculty Mentor			
Team Lead (Student)			
Team Member			
Team Member			

End of Document

This SRS is a living reference for Version 1.0 of the Student Management System. Any material change to scope, roles, or the data model during development should be reflected here via a new revision entry (Document Control, page 3) so the document remains an accurate record for evaluation and future maintenance.